

Construindo código reutilizável e organizado

Módulo 3: Modularidade e Funções

Conceitos fundamentais

Função

Função é um bloco de código que pode ser chamado e executado múltiplas vezes.

Modularidade

Modularidade é a prática de fazer funções de modo que cada uma execute uma tarefa específica e que possa ser reutilizada ao longo do código.

Código **sem** modularidade

```
nota1_aluno1 = 7.5
nota2_aluno1 = 8.0
nota3_aluno1 = 6.5
media_aluno1 = (nota1_aluno1 + nota2_aluno1 + nota3_aluno1) / 3
```

```
SE media_aluno1 >= 7.0 ENTÃO
    ESCRIVER "Aluno 1: APROVADO"
SENÃO
    ESCRIVER "Aluno 1: REPROVADO"
```

```
nota1_aluno2 = 5.0
nota2_aluno2 = 3.0
nota3_aluno2 = 2.5
media_aluno2 = (nota1_aluno2 + nota2_aluno2 + nota3_aluno2) / 3
```

```
SE media_aluno2 >= 7.0 ENTÃO
    ESCRIVER "Aluno 2: APROVADO"
SENÃO
    ESCRIVER "Aluno 2: REPROVADO"
```

```
nota1_aluno3 = 9.5
nota2_aluno3 = 10.0
nota3_aluno3 = 7.5
media_aluno3 = (nota1_aluno3 + nota2_aluno3 + nota3_aluno3) / 3
```

```
SE media_aluno3 >= 7.0 ENTÃO
    ESCRIVER "Aluno 3: APROVADO"
SENÃO
    ESCRIVER "Aluno 3: REPROVADO"
```

Código com modularidade

```
FUNÇÃO calcular_media(nota1, nota2, nota3)  
    media = (nota1 + nota2 + nota3) / 3  
    RETORNAR media
```

```
FUNÇÃO verificar_aprovado(media)  
    SE media >= 7.0 ENTÃO  
        RETORNAR "APROVADO"  
    SENÃO  
        RETORNAR "REPROVADO"
```

Usando essas funções, o código ficaria:

```
media_aluno1 = calcular_media(7.5, 8.0, 6.5)  
ESCREVER verificar_aprovado(media_aluno1)
```

```
media_aluno2 = calcular_media(5.0, 3.0, 2.5)  
ESCREVER verificar_aprovado(media_aluno2)
```

```
media_aluno3 = calcular_media(9.5, 10.0, 7.5)  
ESCREVER verificar_aprovado(media_aluno3)
```

Parâmetros e Retorno de Funções

Talvez você já tenha percebido o padrão da estrutura de uma função. Ela é:

```
nome_da_função (parâmetros):  
    código  
    retorno
```

Parâmetros

Parâmetros são valores que existem FORA da função e que atribuímos às variáveis dentro dela. Veja a função `verificar_aprovado`; seu único parâmetro é a variável `media`, que é usada nos ifs. Passamos para ela, no código principal, a variável `media_aluno1` no lugar de `media`, ou seja, **copiamos** o valor de `media_aluno1` para `media`. Por ser uma cópia, qualquer mudança na variável `media` não altera `media_aluno1`.

Retorno

O **retorno** de uma função é o valor que a função irá gerar ao chegar no fim da execução do código em seu interior. Tipicamente, esse é o valor de uma variável, como na função `calcular_media`. Perceba que no código principal, atribuímos a chamada da função `calcular_media` à variável `media_aluno1`. Isso significa que estamos salvando o valor retornado da função (`media`) nessa variável do código principal.

Detalhes importantes sobre parâmetros e retorno

Normalmente, é preciso definir os tipos dos parâmetros. Como python é fracamente tipado, isso não é necessário.

É possível passar parâmetros por *referência*, de forma que uma mudança na variável dentro de uma função altere a variável passada como parâmetro no código principal, mas não veremos isso aqui.

Em outras linguagens, o tipo da variável que é retornada na função é também o tipo da função, devendo ser explicitado. Pelo mesmo motivo dos parâmetros, python dispensa isso.

❏ **É importante notar que parâmetros e retornos são opcionais.** Uma função que só imprime na tela "Hello World" não precisa nem de parâmetro, nem de retornos.

Funções Próprias

Em python, você pode criar as suas próprias funções usando o operador def e o seguinte formato:

```
def nome_funcao(parametros):  
    # Código  
    return  
  
# Perceba que aqui, apenas declaramos a função. Precisamos chamar ela em  
# algum lugar do código para executá-la.  
def calcular_media(nota1, nota2, nota3):  
    media = (nota1 + nota2 + nota3) / 3  
    return media  
  
media_aluno1 = calcular_media(7.5, 8, 6.5)  
  
print(media_aluno1)
```

Vai ficar mais fácil entender esses conceitos explicando **escopo de variáveis**.

Escopo de variáveis

Escopo é a região do código dentro do qual uma variável é criada. Uma variável só pode ser usada dentro de seu escopo.

Há dois escopos em python (e qualquer linguagem): **global** e **local**.

Escopo Local

O escopo local é aquele dentro de funções. Isso significa que, ao declarar uma variável dentro de uma função, não posso usá-la fora da função:

```
def minha_funcao():  
    z = 100  
# print(z) # NameError: name 'z' is not defined
```

Escopo Global

O escopo global é a parte principal do código. Uma variável criada no escopo global pode ser lida dentro de funções normalmente, mas para modificá-la **dentro** de uma função, há duas formas:

- Por referência (não iremos abordar)
- Usando a palavra-chave global

A palavra-chave global

Lembre-se de que o parâmetro recebe apenas uma cópia do valor da variável, não alterando a variável original no código principal!

A palavra-chave global se refere ao conceito de variável global, ou seja, uma variável que pode ser usada e modificada em qualquer parte do código. No caso de Python, nós não definimos que a variável é global no código principal, mas deixamos claro que a variável dentro da função se refere à variável de mesmo nome no código principal utilizando essa palavra-chave.

📌 **Importante ressaltar caso tenha passado despercebido:** variáveis globais podem ser lidas dentro de funções sem usar global. Ela só é necessária quando você quer modificar a variável global dentro da função.

Exemplo de uso do global

```
contador = 10
```

```
def mostrar_contador():  
    print(contador) # Apenas lê a variável
```

```
mostrar_contador() # 10
```

```
def incrementar():  
    global contador  
    contador = contador + 1
```

```
incrementar()  
print(contador) # 11
```

Funções Built In x Customizáveis

Funções *built-in* são aquelas já definidas e implementadas pelo python, como `print()`, `range()`, as funções de estruturas de dados que vimos e algumas outras. A seguir é uma tabela com todas elas (<https://docs.python.org/pt-br/3.6/library/functions.html>):

Funções Built-in

abs()	dict()	help()	min()	setattr()
all()	dir()	hex()	next()	slice()
any()	divmod()	id()	object()	sorted()
ascii()	enumerate()	input()	oct()	staticmethod()
bin()	eval()	int()	open()	str()
bool()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	
delattr()	hash()	memoryview()	set()	

Documentação e Bibliotecas

Essa é a documentação da função `len()`:

`len(s)`: Retorna o comprimento (o número de itens) de um objeto. O argumento pode ser uma sequência (tal como uma string, bytes, tupla, lista, ou range) ou uma coleção (tal como um dicionário, conjunto, ou conjunto imutável).

(Por ser fracamente tipada, python não tem a melhor documentação do mundo.)

Mas e se eu não encontrar a função que eu quero, ou eu queira uma implementação diferente?

Aí, podemos usar o `def`, ou então, **bibliotecas**. Apesar de não ser o foco desse módulo, bibliotecas são um conjunto de funções e/ou estruturas de dados implementadas por outros usuários que você pode usar no seu código. A maioria (99%) dos códigos em python usam bibliotecas, e vocês verão algumas mais para frente. Para usar uma biblioteca, é preciso baixá-la do terminal usando o comando `pip install nome_da_biblioteca` e a incluir no seu código usando `import nome_da_biblioteca`.

Como usar bibliotecas

Para chamar funções da biblioteca, usamos `nome_da_biblioteca.nome_da_funcao()`. Bibliotecas têm suas próprias documentações, então procure-as!

Podemos mudar o nome de uma biblioteca dentro do nosso código para um nome menor fazendo `import nome_da_biblioteca as nome_curto`, e então fazer só `nome_curto.nome_funcao()`.

matplotlib

Famosa para plotar gráficos

numpy

Facilita usar matrizes e outras operações matemáticas

pandas

Para lidar com dados

scikit learn

Para machine learning

Bibliotecas em python é o que permite seu uso como ferramenta em projetos complexos!

Exemplo de uso de bibliotecas

Aqui vai um rápido exemplo; **não se preocupe em entender as funções**, apenas veja a estrutura do código com o import, etc. A biblioteca matplotlib é famosa para plotar gráficos, e a biblioteca numpy é a ferramenta que citei anteriormente que facilita usar matrizes e outras operações matemáticas. Há outras bibliotecas famosas como pandas para lidar com dados, scikit learn para machine learning e muitas outras.

```
%pip install matplotlib # Terminal
```

```
import matplotlib.pyplot as plt
```

```
meses = ['Jan', 'Fev', 'Mar', 'Abr', 'Mai']
```

```
vendas = [100, 150, 120, 200, 180]
```

```
plt.plot(meses, vendas)
```

```
plt.title('Vendas por Mês')
```

```
plt.xlabel('Mês')
```

```
plt.ylabel('Vendas')
```

```
plt.show()
```

Exercício de Fixação

Sistema de Gerenciamento de Loja

Você foi contratado para criar um sistema completo de gerenciamento de uma pequena loja. O sistema deve ter as seguintes funcionalidades:

01

Cadastro de Produtos

Adicionar novos produtos com: nome, preço e quantidade em estoque

02

Gestão de Vendas

- Registrar uma venda (produto e quantidade)
- Atualizar o estoque automaticamente após a venda
- Calcular o valor total da venda

03

Relatório

Mostrar todos os produtos cadastrados

04

Menu Principal

Interface para o usuário escolher qual operação fazer. Opções:

- Cadastrar produto
- Realizar venda
- Ver relatório
- Sair



Lembre-se: modularidade!

Solução: Funções de Cadastro e Cálculo

```
# FUNÇÕES DE CADASTRO
# Pede os dados e adiciona um produto novo na lista

def cadastrar_produto(produtos):
    print("\n--- Cadastro Produto ---")
    nome = input("Nome do produto: ")
    preco = float(input("Preço: R$ "))
    estoque = int(input("Quantidade em estoque: "))

    # Cria um dicionário com os dados do produto
    novo_produto = {'nome': nome, 'preco': preco, 'estoque': estoque}
    produtos.append(novo_produto)

    print("Produto cadastrado com sucesso")

# Procura um produto na lista pelo nome
def buscar_produto(produtos, nome):
    for produto in produtos:
        if produto['nome'].lower() == nome.lower():
            return produto # Encontrou!

    return None # Não encontrou

# FUNÇÕES DE CÁLCULO
# Multiplica o preço pela quantidade
def calcular_total(preco, quantidade):
    return preco * quantidade
```

```
# FUNÇÕES DE VENDA]
# Faz todo o processo: busca produto, calcula valor, atualiza estoque
def realizar_venda(produtos):
    print("\n--- Realizar Venda ---")
    nome = input("Nome do produto: ")

    # Busca o produto
    produto = buscar_produto(produtos, nome)

    if produto == None:
        print("Produto não encontrado!")
        return

    # Pergunta quantidade
    quantidade = int(input("Quantidade: "))
    # Verificar se tem estoque suficiente
    if quantidade > produto['estoque']:
        print("Estoque insuficiente! Temos apenas", produto['estoque'],
              "unidades")
        return

    # Calcula o valor total
    total = calcular_total(produto['preco'], quantidade)

    # Mostra o resumo da venda
    print("\nProduto:", produto['nome'])
    print("Preço unitário: R$", produto['preco'])
    print("Quantidade:", quantidade)
    print("TOTAL: R$", total)
```

```
# Atualizar o estoque
produto['estoque'] = produto['estoque'] - quantidade

print("\nVenda realizada! Restam", produto['estoque'], "unidades em
estoque")

# FUNÇÕES DE RELATÓRIO
# Mostra a lista de todos os produtos cadastrados
def mostrar_relatorio(produtos):
    print("\n=== RELATÓRIO DE PRODUTOS ===")

    if len(produtos) == 0:
        print("Nenhum produto cadastrado.")
        return

    for produto in produtos:
        print("Produto:", produto['nome'])
        print(" Preço: R$", produto['preco'])
        print(" Estoque:", produto['estoque'], "unidades")
        print()

# FUNÇÕES DE MENU
# Mostra as opções do menu principal
def mostrar_menu():
    print("\nLOJA")
    print("1. Cadastrar produto")
    print("2. Realizar venda")
    print("3. Ver relatório")
    print("4. Sair")
```

```
# PROGRAMA PRINCIPAL
# Começando com alguns produtos de exemplo
produtos = [
    {'nome': 'Arroz', 'preco': 25.0, 'estoque': 50},
    {'nome': 'Feijão', 'preco': 8.5, 'estoque': 30},
    {'nome': 'Macarrão', 'preco': 4.5, 'estoque': 100}
]

# Loop principal - o programa fica rodando até o usuário escolher sair
while True:
    mostrar_menu()
    opcao = input("\nEscolha uma opção: ")

    if opcao == '1':
        cadastrar_produto(produtos)

    elif opcao == '2':
        realizar_venda(produtos)

    elif opcao == '3':
        mostrar_relatorio(produtos)

    elif opcao == '4':
        print("\nEncerrando o sistema")
        break

    else:
        print("Opção inválida, tente novamente")
```